

Structured programming -lab

Course code: cse 1384

Assignment project

Title: Emergency Potable Water Allocation for a Flood-Affected Municipality

Submitted to:

Abdullah Mohammad sakib

lecturer

Department of computer science and technology

Submitted by:

Tawhidul islam

Id:26103042

Program: BCSE

Section: D

Serial :9

Submission date: 6-9-2026



IUBAT- international university of business agricultural and technology

1. Problem Analysis

The dataset supplied for the purpose of this assignment, describing the six service zones and the 13,500 liters of potable water available for distribution among them, is reproduced below.

Zone ID	Service Zone	Type	Requested Water (L)	Minimum Requirement (L)	Distribution Loss (%)	Previous Waiting Cycles
Z01	Municipal Hospital	Hospital	4,000	3,000	2	0
Z02	Central Flood Shelter	Emergency Shelter	3,500	2,500	4	1
Z03	Ward 3 Residential Area	Residential	4,500	2,000	8	2
Z04	Ward 5 Residential Area	Residential	3,800	1,800	12	3
Z05	School Emergency Shelter	Emergency Shelter	2,600	1,600	5	1
Z06	Fire and Emergency Service	Emergency Service	2,000	1,500	1	0

1.1 Nature of the Problem

A flood-affected municipality operating a temporary water-distribution center faces a situation in which the total quantity of potable water requested by its service zones exceeds the quantity actually available for distribution. Because the shortfall cannot be resolved within a single distribution cycle, the center must decide, in a defensible and repeatable manner, how the available water is divided among hospitals, emergency shelters, residential areas and emergency services, each of which differs in the urgency of its need, the length of time it has already waited for supply, and the proportion of released water it is able to actually receive once distribution loss is accounted for.

1.2 Major Factors Affecting the Allocation Decision

- Requested quantity — the volume each zone has asked to receive in the current cycle, representing its stated demand.
- Minimum requirement — the smallest quantity considered adequate for the zone to continue functioning without serious harm; falling below this figure is a materially different outcome from merely receiving less than requested.
- Nature of the service zone — whether the zone is a hospital, an emergency service, an emergency shelter, or a residential area, which determines how immediately a shortfall could endanger life or wellbeing.
- Previous waiting cycles — the number of earlier distribution cycles in which the zone did not receive an adequate share, used as a fairness indicator so that a zone is not repeatedly bypassed.
- Distribution loss — the percentage of water released from the center that is not expected to reach the zone, which means the quantity that must be released is larger than the quantity the zone is meant to effectively receive.

- Available water — the fixed total quantity held at the distribution center at the start of the cycle, which the sum of all allocations must never exceed.
- Unresolved demand — the portion of a zone's requested quantity that remains unmet at any point during the allocation process, which changes as water is progressively allocated.

1.3 Constraints

The total quantity allocated across all six zones must never exceed the 13,500 liters available at the distribution center. Because the minimum requirement of a zone is a quantity that must actually be received rather than merely released, the distribution loss recorded for that zone constrains how much must be released from the center in order for the minimum requirement to be met at the destination; a zone with a high distribution loss requires a proportionally larger release to achieve the same effective delivery as a zone with a low distribution loss. The allocation logic must also remain correct when the supplied requested quantities, minimum requirements, loss percentages, waiting cycles or available quantity are changed, rather than being tied to the specific figures of the dataset above.

1.4 Conflicting Requirements

The engineering considerations stated in the assignment brief identify several pairs of requirements that pull the proposed solution in opposing directions. These are summarized below, and the proposed solution described in Section 3 is designed to hold a reasoned balance between each pair rather than to satisfy one side of a pair at the expense of the other.

Consideration	Conflicting Consideration
Emergency Requirement	Fairness toward zones underserved over several cycles
Complete Supply to a Few Zones	Wider Coverage across all zones
Distribution Efficiency (avoiding high-loss zones)	Social Requirement (the affected population still needs water)
Previous Waiting (long-waiting zones deserve priority)	New Emergency (a sudden urgent need requires immediate attention)
Accuracy of a detailed allocation procedure	Computational Efficiency and simplicity of the program

2. Proposed Solution

2.1 Overview of the Allocation Strategy

The proposed solution allocates the available water in two ordered phases rather than in the order in which the zones happen to be listed. In the first phase, the available water is used to bring every zone, in priority order, as close as possible to its minimum requirement after distribution loss is accounted for. In the second phase, any water remaining after every zone has been considered for its minimum requirement is used, again in priority order, to move zones progressively from their minimum requirement

toward their full requested quantity. This two-phase structure directly reflects the reasoning that a dependable minimum supply to every zone, particularly hospitals and emergency services, is a more urgent objective than fully satisfying the request of a smaller number of zones while others remain below a safe minimum.

2.2 Determining Allocation Priority

Before either allocation phase begins, every zone is assigned a priority score that combines the following factors.

Factor	Symbol	Role in the Allocation Priority
Zone Type Criticality	T	Hospitals and emergency services are assigned the highest criticality weight because an interruption in their supply carries an immediate risk to life; emergency shelters, which house displaced people, are assigned the next highest weight; residential zones are assigned a base weight, since a brief delay, while undesirable, does not typically carry the same immediate risk.
Minimum-Requirement Urgency	M	Computed from how large the minimum requirement is relative to the requested quantity, and from whether the minimum requirement has already been met in the current cycle; a zone whose minimum requirement has not yet been met contributes more strongly to the score than one that has already secured it.
Previous Waiting Cycles	W	A zone that has already been underserved for a greater number of previous distribution cycles contributes a larger fairness component to the score, so that a zone is not repeatedly bypassed simply because it is, in isolation, less critical than another.
Distribution Loss	L	Not used to raise or lower a zone's priority ranking directly (see Section 3.4), but used afterwards to determine how much water must actually be released to that zone in order to deliver a given effective quantity.
Unresolved Demand	U	The portion of a zone's requested quantity not yet allocated; recalculated after every allocation step and used to decide whether a zone still requires further water during the surplus-distribution phase.

The zone-type criticality component ensures that, all else being equal, a hospital or emergency service is considered ahead of a residential zone. The waiting-cycle component ensures that a zone which has already gone without adequate supply in previous cycles is not indefinitely overtaken by zones of nominally higher criticality, so that the fairness concern raised in the assignment brief is represented explicitly rather than left unaddressed. The minimum-requirement urgency component ensures that a zone whose minimum requirement is large relative to its request, or has not yet been met, is not ranked below a zone that is already comfortably served. Once computed, the priority score of a zone is stored with its record and reused throughout both allocation phases, rather than being recalculated at every step.

2.3 Equal-Priority Resolution

Where two or more zones produce an identical, or near-identical, combined priority score, the proposed solution resolves the tie through the following ordered sequence of secondary comparisons: first, the zone of higher type criticality (hospital or emergency service, then emergency shelter, then residential) is placed ahead; second, if the type criticality is also equal, the zone with the greater number of previous waiting cycles is placed ahead, since it has gone longer without adequate supply; and third, if the waiting cycles are also equal, the zone with the larger minimum requirement still unmet is placed ahead, since it has the greater immediate need. This sequence is deterministic and uses only values already available from the priority-scoring step, so no additional data collection or recalculation is required to resolve a tie.

2.4 Treatment of Distribution Loss

Distribution loss is deliberately not included as a factor that raises or lowers a zone's position in the priority order, since doing so would penalize a zone for a physical characteristic of its supply route rather than for any reduction in its genuine need, which the assignment brief specifically warns against. Instead, distribution loss is applied at the point where a target effective quantity (whether the minimum requirement, in the first phase, or a further share of the requested quantity, in the second phase) is converted into the quantity that must actually be released from the distribution center, calculated as the target quantity divided by the fraction of water expected to survive the loss. This keeps the ranking of zones governed by genuine need and fairness, while still ensuring that a high-loss zone is not effectively under-supplied at its destination merely because its loss percentage was ignored during release planning.

3. Program Design

3.1 Data Organization

Each service zone is represented as a single structured record containing its zone ID, service-zone name, type, requested water, minimum requirement, distribution-loss percentage and previous waiting cycles as supplied, together with derived fields for its priority score, water allocated, expected distribution loss in liters, effective quantity delivered, remaining shortage and final service condition. All six (or, in general, any number of) zone records are held in a single array of such structures. Keeping the derived fields inside the same record as the supplied fields avoids separate parallel arrays and ensures that every value belonging to a zone is carried together whenever the array is ordered, searched or updated, which also satisfies the requirement that related values remain correctly associated with one another.

3.2 Major Functions

The proposed solution is organized into a set of functions, each responsible for one aspect of the overall task, so that operations such as computing an effective delivery or classifying a service condition are implemented once and reused for every zone rather than repeated inline. The intended responsibility of each function is summarized below the corresponding source code is presented separately as part of the submitted implementation.

Proposed Function (Purpose)	Responsibility
Data initialization routine	Loads the six service-zone records (zone ID, service zone name, type, requested water, minimum requirement, distribution loss and previous waiting cycles) into the chosen data structure at start-up, from in-code initial values, so that the program can equally be exercised on a dataset of a different size or with different values.

Proposed Function (Purpose)	Responsibility
Priority evaluator	Computes, for every zone, the criticality, minimum-requirement urgency and waiting-cycle components described in Section 3.3, combines them into a single priority score, and stores the score in the record so that it is available to every later step without being recalculated.
Ordering routine	Arranges the zone records in descending order of priority score, applying the equal-priority resolution sequence of Section 3.4 where required, while keeping every field of a record correctly associated with it during rearrangement.
Minimum-guarantee allocator	Iterates over the zones in priority order and releases, to each zone in turn, the quantity of water required to deliver its minimum requirement after accounting for its distribution loss, limited by whatever quantity remains available at the distribution center.
Surplus allocator	After the minimum-guarantee pass, iterates over the zones in priority order again and releases further water toward each zone's remaining requested quantity, again accounting for distribution loss, until either every zone's request is satisfied or the available water is exhausted.
Effective-delivery calculator	Derives, for each zone, the expected distribution loss in liters and the effective quantity delivered, from the water allocated to that zone and its recorded loss percentage, and derives the remaining shortage as the difference between the requested quantity and the effective quantity delivered.
Service-condition classifier	Assigns each zone one of the defined service conditions (full requirement satisfied, minimum requirement satisfied, below minimum requirement, or unserved) by comparing the effective quantity delivered against the requested quantity and the minimum requirement.
Search routine	Accepts an identifying value (the zone ID, or alternatively the service-zone name) and locates the corresponding record, reporting an appropriate message when no match exists.
Highest-unresolved-requirement finder	Scans the zone records once to identify the zone with the greatest remaining shortage, reusing the shortage values already derived by the effective-delivery calculator rather than recalculating them.
Summary and report generator	Produces the organized per-zone summary and the aggregate totals (total requested, total allocated, total effective delivered, total unresolved requirement, remaining water at the center, and the counts of zones in each service condition) in a single pass over the (ordered) array.
Menu / control routine	Presents the available operations to the user and calls the corresponding routine, allowing the program to be exercised repeatedly, including with modified input values, during demonstration.

3.3 Service Condition Classification

Once allocation is complete, every zone is assigned one of the following service conditions, determined from the effective quantity it received relative to its requested quantity and its minimum requirement.

Service Condition	Governing Situation
Full Requirement Satisfied	The effective quantity delivered to the zone is equal to (or, allowing for rounding, not less than) its full requested quantity.
Minimum Requirement Satisfied	The effective quantity delivered is less than the requested quantity but is at least equal to the zone's minimum requirement.
Below Minimum Requirement	Some water has been allocated and delivered to the zone, but the effective quantity delivered remains less than its minimum requirement.
Unserved	No water has been allocated to the zone in the current distribution cycle, so its effective quantity delivered is zero.

3.4 Algorithm / Pseudocode

The overall control flow of the proposed program, expressed at a level intended to convey the reasoning of the solution rather than the C-language syntax of its implementation, proceeds as follows.

1. Load the supplied service-zone records into the array of structures described in Section 4.1, together with the total available water at the distribution center.
2. For every record, in a single pass, compute its priority score from the zone-type criticality, minimum-requirement urgency and waiting-cycle components described in Section 3.2, and store the score in the record.
3. Arrange the array in descending order of priority score, applying the equal-priority resolution sequence of Section 3.3 where required.
4. Minimum-guarantee phase: for each zone, in the ordered sequence, calculate the release quantity required to deliver its minimum requirement after distribution loss; allocate the smaller of this required release and whatever water currently remains available; reduce the remaining available water by the quantity allocated; and stop this phase immediately, for any remaining zones, once the available water reaches zero.
5. Surplus-distribution phase: if water still remains after the minimum-guarantee phase, repeat a pass over the zones in the same priority order, allocating further water toward each zone's remaining requested quantity (again converted to a release quantity through its distribution loss), continuing until either every zone's request is fully met or the available water reaches zero.
6. For every zone, once its final allocated quantity is known, compute the expected distribution loss in liters, the effective quantity delivered, and the remaining shortage, and assign its service condition using the rules in Section 4.3.
7. Present the user with a menu offering: search for a zone, view the zones ordered by priority or another selected criterion, identify the zone with the highest unresolved requirement, and generate the final summary.

8. If the user selects the search operation, accept an identifying value, examine the array for a matching record, and display its full stored and derived information, or an appropriate not-found message if no match exists.
9. If the user selects the summary operation, iterate over the (optionally ordered) array once, displaying the required per-zone fields and accumulating, in the same pass, the aggregate totals and condition counts required by the assignment brief.
10. Return to the menu until the user chooses to end the program.

3.5 Flowchart Description

The corresponding flow proceeds, at a high level, from program start, through the one-time loading, priority-scoring and ordering steps described in Steps 1 to 3 above, into the two-phase allocation described in Steps 4 and 5, which together determine every zone's final allocated quantity before any output is produced. Control then enters a repeating menu loop offering the search, priority-view, highest-shortage and summary operations as separate branches, each of which reuses the allocation results already computed rather than repeating the allocation process. The loop terminates only when the user selects the exit option.

4. Optimization Considerations

The proposed design applies the optimization requirement of the assignment in the following specific ways.

- Single computation of the priority score: each zone's priority score is calculated exactly once, immediately after the data is loaded, and stored within the record itself, rather than being recalculated whenever zones are compared during ordering or during either allocation phase.
- Two bounded allocation passes rather than an unbounded search: the minimum-guarantee and surplus-distribution phases each make at most one ordered pass over the array (with the surplus phase repeating only while water remains and at least one zone is still unresolved), rather than repeatedly re-scanning the full dataset to decide the next recipient.
- Reuse of the release/effective-delivery relationship: the conversion between a target effective quantity and the corresponding release quantity, and between an allocated quantity and its effective delivery, is implemented once and applied to whichever quantity is relevant at that step, rather than being derived separately in each phase.
- Single-pass summary and counting: the aggregate totals and the counts of zones in each service condition are accumulated during the same pass over the array that produces the per-zone summary lines, rather than in a separate pass.
- Early termination once water is exhausted: both allocation phases stop advancing through the remaining zones as soon as the available water reaches zero, rather than continuing to iterate over zones that cannot receive any further allocation.
- Selection of a sorting technique proportionate to the dataset: given the small number of service-zone records expected in this setting, a straightforward comparison-based sorting technique is considered sufficient and is preferred over a more elaborate technique whose additional complexity would not be justified by the size of the dataset.

5. code

The screenshot shows the Code::Blocks IDE with the file '1384==.c' open. The code defines a 'Zone' struct and lists function prototypes. The struct 'Zone' contains fields for id, name, type, requested, minimum, loss, waiting, priority, order, allocated, lossamount, delivered, shortage, and status. The function prototypes include loadOriginalData, resetResults, geturgency, calculatePriority, sortZones, allocateWater, calculateResults, runallocation, displayResult, searchZone, and findHighestShortage.

```
1 #include <stdio.h>
2 #include <string.h>
3 #define TOTAL_ZONES 6
4 struct Zone
5 {
6     char id[10];
7     char name[40];
8     char type[25];
9     float requested;
10    float minimum;
11    float loss;
12    int waiting;
13    float priority;
14    int order;
15    float allocated;
16    float lossamount;
17    float delivered;
18    float shortage;
19    char status[20];
20 };
21 /* Function Prototypes */
22 void loadOriginalData(struct Zone zone[]);
23 void resetResults(struct Zone zone[]);
24 float geturgency(char type[]);
25 void calculatePriority(struct Zone zone[]);
26 void sortZones(struct Zone zone[]);
27 void allocateWater(struct Zone zone[], float available);
28 void calculateResults(struct Zone zone[]);
29 void runallocation(struct Zone zone[], float available);
30 void displayResult(struct Zone zone[], float available);
31 void searchZone(struct Zone zone[]);
32 void findHighestShortage(struct Zone zone[]);
```

The screenshot shows the continuation of the C code in '1384==.c'. It includes the implementation of the 'loadOriginalData' function, which initializes an array of 'Zone' structures with specific data for three zones: 'Municipal Hospital', 'Central Flood Shelter', and 'Emergency Shelter'.

```
31 void searchZone(struct Zone zone[]);
32 void findHighestShortage(struct Zone zone[]);
33 void case1(struct Zone original[]);
34 void case2(struct Zone original[]);
35 void case3(struct Zone original[]);
36 void case4(struct Zone original[]);
37 void case5(struct Zone original[]);
38 void case6(struct Zone original[]);
39 void case7(struct Zone original[]);
40
41 /* LOAD ORIGINAL DATA
42 ===== */
43 void loadOriginalData(struct Zone zone[])
44 {
45     /* 201 */
46     strcpy(zone[0].id, "201");
47     strcpy(zone[0].name, "Municipal Hospital");
48     strcpy(zone[0].type, "Hospital");
49     zone[0].requested = 4000;
50     zone[0].minimum = 3000;
51     zone[0].loss = 2;
52     zone[0].waiting = 0;
53     /* 202 */
54     strcpy(zone[1].id, "202");
55     strcpy(zone[1].name, "Central Flood Shelter");
56     strcpy(zone[1].type, "Emergency Shelter");
57     zone[1].requested = 3500;
58     zone[1].minimum = 2500;
59     zone[1].loss = 4;
60     zone[1].waiting = 1;
61     /* 203 */
62     strcpy(zone[2].id, "203");
```

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management Projects Symbol Workspace

```
61 /* 203 */
62 strcpy(zone[2].id, "203");
63 strcpy(zone[2].name, "Ward 3");
64 strcpy(zone[2].type, "Residential");
65 zone[2].requested = 4500;
66 zone[2].minimum = 2000;
67 zone[2].loss = 8;
68 zone[2].waiting = 2;
69 /* 204 */
70 strcpy(zone[3].id, "204");
71 strcpy(zone[3].name, "Ward 5");
72 strcpy(zone[3].type, "Residential");
73 zone[3].requested = 3800;
74 zone[3].minimum = 1800;
75 zone[3].loss = 12;
76 zone[3].waiting = 3;
77 /* 205 */
78 strcpy(zone[4].id, "205");
79 strcpy(zone[4].name, "School Emergency Shelter");
80 strcpy(zone[4].type, "Emergency Shelter");
81 zone[4].requested = 2600;
82 zone[4].minimum = 1600;
83 zone[4].loss = 5;
84 zone[4].waiting = 1;
85 /* 206 */
86 strcpy(zone[5].id, "206");
87 strcpy(zone[5].name, "Fire and Emergency Service");
88 strcpy(zone[5].type, "Emergency Service");
89 zone[5].requested = 2000;
90 zone[5].minimum = 1500;
91 zone[5].loss = 1;
92 zone[5].waiting = 0;
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management Projects Symbol Workspace

```
91 zone[5].loss = 1;
92 zone[5].waiting = 0;
93 resetResults(zone);
94 }
95 /*
96  RESET CALCULATED VALUES
97  */
98 void resetResults(struct Zone zone[])
99 {
100     int i;
101     for (i = 0; i < TOTAL_ZONES; i++)
102     {
103         zone[i].priority = 0;
104         zone[i].order = 0;
105         zone[i].allocated = 0;
106         zone[i].lossAmount = 0;
107         zone[i].delivered = 0;
108         zone[i].shortage = 0;
109         strcpy(zone[i].status, "Unprocessed");
110     }
111 }
112 /*
113  URGENCY SCORE
114  */
115 float getUrgency(char type[])
116 {
117     if (strcmp(type, "Hospital") == 0)
118         return 10;
119     if (strcmp(type, "Emergency Service") == 0)
120         return 10;
121     if (strcmp(type, "Emergency Shelter") == 0)
122         return 0;
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main(): int

Management Projects Symbol Workspace

```
121 if (strcmp(type, "Emergency Shelter") == 0)
122     return 5;
123 if (strcmp(type, "Residential") == 0)
124     return 5;
125     return 0;
126 }
127 /*
128  PRIORITY CALCULATION
129  Proposed strategy:
130  Priority =
131  Urgency
132  + Waiting Score
133  + Unresolved Demand Score
134  - Loss Penalty
135  This is our proposed strategy because the assignment
136  does not provide a fixed formula.
137  ===== */
138 void calculatePriority(struct Zone zone[])
139 {
140     int i;
141     for (i = 0; i < TOTAL_ZONES; i++)
142     {
143         float urgency;
144         float waitingScore;
145         float demandScore;
146         float lossPenalty;
147         urgency = getUrgency(zone[i].type);
148         /* None waiting = none priority */
149         waitingScore = zone[i].waiting * 2.0;
150         /* Larger gap between request and minimum = more need */
151         demandScore =
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main(): int

Management Projects Symbol Workspace

```
151 /* Larger gap between request and minimum = more need */
152 demandScore =
153     (zone[i].requested - zone[i].minimum) / 500.0;
154 /* Higher loss gets a small penalty */
155 lossPenalty = zone[i].loss * 0.10;
156 zone[i].priority =
157     urgency +
158     waitingScore +
159     demandScore -
160     lossPenalty;
161 }
162 }
163 /*
164  SORT ZONES BY PRIORITY
165  ===== */
166 void sortZones(struct Zone zone[])
167 {
168     int i, j;
169     struct Zone temp;
170     for (i = 0; i < TOTAL_ZONES - 1; i++)
171     {
172         for (j = 0; j < TOTAL_ZONES - i - 1; j++)
173         {
174             if (zone[j].priority < zone[j + 1].priority)
175             {
176                 temp = zone[j];
177                 zone[j] = zone[j + 1];
178                 zone[j + 1] = temp;
179             }
180             /* Tie-breaking rule */
181             else if (zone[j].priority == zone[j + 1].priority)
182             {
```

Logs & others

\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global> main(): int
181 else if (zone[j].priority == zone[j + 1].priority)
182 {
183 /* First compare waiting cycles */
184 if (zone[j].waiting < zone[j + 1].waiting)
185 {
186 temp = zone[j];
187 zone[j] = zone[j + 1];
188 zone[j + 1] = temp;
189 }
190 /* If waiting is also equal, compare minimum */
191 else if (zone[j].waiting == zone[j + 1].waiting)
192 {
193 if (zone[j].minimum < zone[j + 1].minimum)
194 {
195 temp = zone[j];
196 zone[j] = zone[j + 1];
197 zone[j + 1] = temp;
198 }
199 }
200 }
201 }
202 for (i = 0; i < TOTAL_ZONES; i++)
203 {
204 zone[i].order = i + 1;
205 }
206 }
207 }
208 /*
209 WATER ALLOCATION
210 */
211 void allocateWater(struct Zone zone[], float available)
212 {

Logs & others

\\Users\\user\\OneDrive\\Documents\\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
<global> main(): int
211 void allocateWater(struct Zone zone[], float available)
212 {
213 int i;
214 for (i = 0; i < TOTAL_ZONES; i++)
215 {
216 float fullRequirement;
217 float minimumRequirement;
218 if (available <= 0)
219 {
220 zone[i].allocated = 0;
221 continue;
222 }
223 /*
224 Because of distribution loss, the center must release
225 more water than the destination finally receives.
226 Example:
227 Requested = 4000 L
228 Loss = 2%
229 Release required to deliver 4000 L:
230 4000 / 0.98
231 */
232 fullRequirement =
233 zone[i].requested /
234 (1.0 - zone[i].loss / 100.0);
235 minimumRequirement =
236 zone[i].minimum /
237 (1.0 - zone[i].loss / 100.0);
238 /*
239 Try to satisfy the complete request first.
240 */
241 if (available >= fullRequirement)
242 {

Logs & others

\\Users\\user\\OneDrive\\Documents\\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main(): int

Management Projects Symbol Workspace

```
241     if (available >= fullRequirement)
242     {
243         zone[i].allocated = fullRequirement;
244         available = available - fullRequirement;
245     }
246     /*
247     If complete request is impossible,
248     try to satisfy the minimum requirement.
249     */
250     else if (available >= minimumRequirement)
251     {
252         zone[i].allocated = minimumRequirement;
253         available = available - minimumRequirement;
254     }
255     /*
256     If even the minimum cannot be supplied,
257     give the remaining water.
258     */
259     else
260     {
261         zone[i].allocated = available;
262         available = 0;
263     }
264 }
265 /*
266 =====
267 CALCULATE LOSS, DELIVERY AND SHORTAGE
268 ===== */
269 void calculateResults(struct Zone zone[])
270 {
271     int i;
272     for (i = 0; i < TOTAL_ZONES; i++)
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main(): int

Management Projects Symbol Workspace

```
271     int i;
272     for (i = 0; i < TOTAL_ZONES; i++)
273     {
274         /* Calculate distribution loss */
275         zone[i].lossAmount =
276             zone[i].allocated * zone[i].loss / 100.0;
277         /* Calculate actual delivered water */
278         zone[i].delivered =
279             zone[i].allocated - zone[i].lossAmount;
280         /* Calculate remaining shortage */
281         zone[i].shortage =
282             zone[i].requested - zone[i].delivered;
283         if (zone[i].shortage < 0)
284             zone[i].shortage = 0;
285         /* Determine service condition */
286         if (zone[i].delivered >= zone[i].requested)
287         {
288             strcpy(zone[i].status, "FULL");
289         }
290         else if (zone[i].delivered >= zone[i].minimum)
291         {
292             strcpy(zone[i].status, "MINIMUM");
293         }
294         else if (zone[i].delivered > 0)
295         {
296             strcpy(zone[i].status, "BELOW MIN");
297         }
298         else
299         {
300             strcpy(zone[i].status, "UNSERVED");
301         }
302     }
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management Projects Symbol Workspace

Start here 1384==.c

```
301 }
302 }
303 }
304 /*
305  * COMPLETE ALLOCATION PROCESS
306  */
307 void runAllocation(struct Zone zone[], float available)
308 {
309     resetResults(zone);
310     calculatePriority(zone);
311     sortZones(zone);
312     allocateWater(zone, available);
313     calculateResults(zone);
314 }
315 /*
316  * DISPLAY RESULT
317  */
318 void displayResult(struct Zone zone[], float available)
319 {
320     int i;
321     float totalRequested = 0;
322     float totalAllocated = 0;
323     float totalDelivered = 0;
324     float totalShortage = 0;
325     int full = 0;
326     int minimum = 0;
327     int belowMinimum = 0;
328     int unserved = 0;
329     printf("\n\n");
330     printf("===== \n");
331     printf("ALLOCATION RESULT\n");
332     printf("===== \n");
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management Projects Symbol Workspace

Start here 1384==.c

```
331 printf("ALLOCATION RESULT\n");
332 printf("===== \n");
333 printf("%-5s %-14s %-9s %-9s %-9s %-10s %-10s %-10s %-10s\n",
334     "ID",
335     "Zone",
336     "Request",
337     "Minimum",
338     "Priority",
339     "Allocated",
340     "Delivered",
341     "Shortage",
342     "Status");
343 printf("----- \n");
344 for (i = 0; i < TOTAL_ZONES; i++)
345 {
346     printf("%-5s %-14s %-9.0f %-9.0f %-9.2f %-10.0f %-10.0f %-10.0f %-10s\n",
347         zone[i].id,
348         zone[i].name,
349         zone[i].requested,
350         zone[i].minimum,
351         zone[i].priority,
352         zone[i].allocated,
353         zone[i].delivered,
354         zone[i].shortage,
355         zone[i].status);
356     totalRequested += zone[i].requested;
357     totalAllocated += zone[i].allocated;
358     totalDelivered += zone[i].delivered;
359     totalShortage += zone[i].shortage;
360     if (strcmp(zone[i].status, "FULL") == 0)
361         full++;
362     else if (strcmp(zone[i].status, "MINIMUM") == 0)
```

Logs & others

\\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Start here x 1384==.c x

```
361 full++;
362 else if (strcmp(zone[i].status, "MINIMUM") == 0)
363     minimum++;
364 else if (strcmp(zone[i].status, "BELOW MIN") == 0)
365     belowMinimum++;
366 else if (strcmp(zone[i].status, "UNSERVED") == 0)
367     unserved++;
368 }
369 printf("===== \n");
370 printf("\nPriority Order: \n");
371 for (i = 0; i < TOTAL_ZONES; i++)
372 {
373     printf("%d. %s - %s (Priority %.2f) \n",
374           zone[i].order,
375           zone[i].id,
376           zone[i].name,
377           zone[i].priority);
378 }
379 printf("\nSummary \n");
380 printf("----- \n");
381 printf("Total Requested Water : %.2f \n",
382       totalRequested);
383 printf("Total Allocated Water : %.2f \n",
384       totalAllocated);
385 printf("Total Effective Water Delivered : %.2f \n",
386       totalDelivered);
387 printf("Total Unresolved Requirement : %.2f \n",
388       totalShortage);
389 printf("Remaining Water : %.2f \n",
390       available - totalAllocated);
391 printf("\nService Conditions \n");
392 printf("----- \n");
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Start here x 1384==.c x

```
391 printf("\nService Conditions \n");
392 printf("----- \n");
393 printf("Full Requirement Satisfied : %d zone(s) \n",
394       full);
395 printf("Minimum Requirement Satisfied: %d zone(s) \n",
396       minimum);
397 printf("Below Minimum : %d zone(s) \n",
398       belowMinimum);
399 printf("Completely Unserved : %d zone(s) \n",
400       unserved);
401 }
402 /*
403 SEARCH A ZONE
404 */
405 void searchZone(struct Zone zone[])
406 {
407     char id[10];
408     int i;
409     int found = 0;
410     printf("\nEnter Zone ID: ");
411     scanf("%s", id);
412     for (i = 0; i < TOTAL_ZONES; i++)
413     {
414         if (strcmp(zone[i].id, id) == 0)
415         {
416             printf("\nZone Found \n");
417             printf("----- \n");
418             printf("Zone ID : %s \n", zone[i].id);
419             printf("Service Zone : %s \n", zone[i].name);
420             printf("Type : %s \n", zone[i].type);
421             printf("Requested : %.2f \n", zone[i].requested);
422             printf("Minimum : %.2f \n", zone[i].minimum);
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management x

Projects Symbol

Workspace

```
421     printf("Requested      : %.2f L\n", zone[i].requested);
422     printf("Minimum       : %.2f L\n", zone[i].minimum);
423     printf("Loss           : %.2f %s\n", zone[i].loss);
424     printf("Waiting Cycles   : %d\n", zone[i].waiting);
425     printf("Priority         : %.2f\n", zone[i].priority);
426     printf("Allocated        : %.2f L\n", zone[i].allocated);
427     printf("Loss Amount       : %.2f L\n", zone[i].lossAmount);
428     printf("Effective Water   : %.2f L\n", zone[i].delivered);
429     printf("Shortage          : %.2f L\n", zone[i].shortage);
430     printf("Condition        : %s\n", zone[i].status);
431     found = 1;
432     break;
433 }
434 }
435 if (found == 0)
436     printf("\nNone not found.\n");
437 }
438
439 /*
440  FIND HIGHEST UNRESOLVED REQUIREMENT
441 */
442 void findHighestShortage(struct Zone zone[])
443 {
444     int i;
445     int position = 0;
446     for (i = 1; i < TOTAL_ZONES; i++)
447     {
448         if (zone[i].shortage > zone[position].shortage)
449         {
450             position = i;
451         }
452     }
453     printf("\nHighest Unresolved Requirement\n");
454 }
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management x

Projects Symbol

Workspace

```
451 }
452 printf("\nHighest Unresolved Requirement\n");
453 printf("-----\n");
454 printf("Zone ID : %s\n", zone[position].id);
455 printf("Zone    : %s\n", zone[position].name);
456 printf("Shortage : %.2f L\n", zone[position].shortage);
457 }
458
459 /*
460  CASE 1
461  Original dataset + 13,500 L
462 */
463 void case1(struct Zone original[])
464 {
465     struct Zone zone[TOTAL_ZONES];
466     int i;
467     for (i = 0; i < TOTAL_ZONES; i++)
468     {
469         zone[i] = original[i];
470         printf("\n===== CASE 1 =====\n");
471         printf("Original Dataset\n");
472         printf("Available Water: 13,500 L\n");
473         runAllocation(zone, 13500);
474         displayResult(zone, 13500);
475     }
476
477 /*
478  CASE 2
479  Enough water for all zones
480 */
481 void case2(struct Zone original[])
482 {
483     struct Zone zone[TOTAL_ZONES];
484     int i;
485     float enoughWater = 0;
486 }
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management Projects Symbol Workspace

```
481 int i;
482 float enoughWater = 0;
483 for (i = 0; i < TOTAL_ZONES; i++)
484 {
485     /*
486      * Calculate the amount required from the zones
487      * to deliver the complete requested quantity
488      * after distribution loss.
489      */
490     enoughWater +=
491         original[i].requested /
492         (1.0 - original[i].loss / 100.0);
493 }
494 /*
495 * Add a little extra so there is definitely enough.
496 */
497 enoughWater += 500;
498 for (i = 0; i < TOTAL_ZONES; i++)
499     zone[i] = original[i];
500 printf("\n===== CASE 2 =====\n");
501 printf("Enough Water for Complete Requested Quantity\n");
502 printf("Available Water: %.2f l\n", enoughWater);
503 runAllocation(zone, enoughWater);
504 displayResult(zone, enoughWater);
505 }
506 /*
507 * CASE 3
508 * User gives water below combined minimum requirement
509 * ===== */
510 void case3(struct Zone original[])
511 {
512     struct Zone zone[TOTAL_ZONES];
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management Projects Symbol Workspace

```
511 {
512     struct Zone zone[TOTAL_ZONES];
513     int i;
514     float available;
515     for (i = 0; i < TOTAL_ZONES; i++)
516         zone[i] = original[i];
517     printf("\n===== CASE 3 =====\n");
518     printf("Enter available water in litres: ");
519     scanf("%f", &available);
520     if (available <= 0)
521     {
522         printf("Water quantity must be greater than zero.\n");
523         return;
524     }
525     runAllocation(zone, available);
526     displayResult(zone, available);
527 }
528 /*
529 * CASE 4
530 * Equal priority
531 * ===== */
532 void case4(struct Zone original[])
533 {
534     struct Zone zone[TOTAL_ZONES];
535     int i;
536     for (i = 0; i < TOTAL_ZONES; i++)
537         zone[i] = original[i];
538     /*
539      * Modify 203 and 204 so that their priority
540      * becomes equal.
541      */
542     zone[2].waiting = 3;
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

82° 9:38 PM 9/4/2026

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management Projects Symbol Workspace

```
541 /*
542 zone[2].waiting = 3;
543 zone[3].waiting = 3;
544 zone[2].requested = 4000;
545 zone[3].requested = 4000;
546 zone[2].minimum = 2000;
547 zone[3].minimum = 2000;
548 zone[2].loss = 10;
549 zone[3].loss = 10;
550 printf("===== CASE 4 =====\n");
551 printf("Equal Priority Test\n");
552 printf("Z03 and Z04 have been modified to create equal priority.\n");
553 runAllocation(zone, 13500);
554 displayResult(zone, 13500);
555 */
556
557 CASE 5
558 User increases loss of a residential zone
559
560 void case5(struct Zone original[])
561 {
562     struct Zone zone[TOTAL_ZONES];
563     int i;
564     int position = -1;
565     char id[10];
566     float newLoss;
567     for (i = 0; i < TOTAL_ZONES; i++)
568         zone[i] = original[i];
569     printf("===== CASE 5 =====\n");
570     printf("Residential zones are: Z03 and Z04\n");
571     printf("Enter residential Zone ID: ");
572     scanf("%s", id);
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management Projects Symbol Workspace

```
571 printf("Enter residential Zone ID: ");
572 scanf("%s", id);
573 /* Find the selected zone */
574 for (i = 0; i < TOTAL_ZONES; i++)
575 {
576     if (strcmp(zone[i].id, id) == 0)
577     {
578         position = i;
579         break;
580     }
581 }
582 if (position == -1)
583 {
584     printf("Zone not found.\n");
585     return;
586 }
587 /* Check whether it is residential */
588 if (strcmp(zone[position].type, "Residential") != 0)
589 {
590     printf("Invalid zone. Please select a residential zone.\n");
591     return;
592 }
593 printf("Current distribution loss: %.2f%%\n",
594        zone[position].loss);
595 printf("Enter new distribution loss (%): ");
596 scanf("%f", &newLoss);
597 if (newLoss < 0 || newLoss >= 100)
598 {
599     printf("Invalid loss percentage.\n");
600     return;
601 }
602 zone[position].loss = newLoss;
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management Projects Symbol Workspace

```
601 }
602 zone[position].loss = newLoss;
603 printf("\nNew loss of %s = %.2f%%\n",
604        zone[position].id,
605        zone[position].loss);
606 runAllocation(zone, 13500);
607 displayResult(zone, 13500);
608 }
609 /*
610 CASE 6
611 User increases waiting period of an underserved zone
612 ===== */
613 void case6(struct Zone original[])
614 {
615     struct Zone zone[TOTAL_ZONES];
616     int i;
617     int position = -1;
618     char id[10];
619     int newWaiting;
620     /*
621     First run original case to identify underserved zones.
622     */
623     for (i = 0; i < TOTAL_ZONES; i++)
624         zone[i] = original[i];
625     runAllocation(zone, 13500);
626     printf("\n===== CASE 6 =====\n");
627     printf("Previously underserved zones from Case 1:\n");
628     for (i = 0; i < TOTAL_ZONES; i++)
629     {
630         if (strcmp(zone[i].status, "BELOW MIN") == 0 ||
631             strcmp(zone[i].status, "UNSERVED") == 0)
632         {
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management Projects Symbol Workspace

```
631     strcmp(zone[i].status, "UNSERVED") == 0)
632     {
633         printf("%s - %s\n",
634                zone[i].id,
635                zone[i].name);
636     }
637     printf("\nEnter Zone ID from the underserved list: ");
638     scanf("%s", id);
639     for (i = 0; i < TOTAL_ZONES; i++)
640     {
641         if (strcmp(zone[i].id, id) == 0)
642         {
643             position = i;
644             break;
645         }
646     }
647     if (position == -1)
648     {
649         printf("Zone not found.\n");
650         return;
651     }
652     /*
653     Check whether selected zone was underserved.
654     */
655     if (strcmp(zone[position].status, "BELOW MIN") != 0 &&
656         strcmp(zone[position].status, "UNSERVED") != 0)
657     {
658         printf("This zone was not underserved in Case 1.\n");
659         return;
660     }
661     printf("Current waiting cycles: %d\n",
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management Projects Symbol Workspace

Start here 1384==.c

```
661 }
662 printf("Current waiting cycles: %d\n",
663       zone[position].waiting);
664 printf("Enter new waiting cycles: ");
665 scanf("%d", &newWaiting);
666 if (newWaiting < zone[position].waiting)
667 {
668     printf("New waiting cycle should be greater than the current value.\n");
669     return;
670 }
671 zone[position].waiting = newWaiting;
672 printf("\nWaiting cycles of %s changed to %d.\n",
673       zone[position].id,
674       zone[position].waiting);
675 runAllocation(zone, 13500);
676 displayResult(zone, 13500);
677 }
678 /* =====
679 CASE 7
680 Independent test - user chooses water amount
681 ===== */
682 void case7(struct Zone original[])
683 {
684     struct Zone zone[TOTAL_ZONES];
685     int i;
686     float available;
687     for (i = 0; i < TOTAL_ZONES; i++)
688         zone[i] = original[i];
689     printf("\n===== CASE 7 =====\n");
690     printf("Independent Test Case\n");
691     printf("Enter any available water quantity: ");
692     scanf("%f", &available);
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management Projects Symbol Workspace

Start here 1384==.c

```
691 printf("Enter any available water quantity: ");
692 scanf("%f", &available);
693 if (available <= 0)
694 {
695     printf("Water quantity must be greater than zero.\n");
696     return;
697 }
698 runAllocation(zone, available);
699 displayResult(zone, available);
700 }
701 /* =====
702 MAIN FUNCTION
703 ===== */
704 int main()
705 {
706     struct Zone original[TOTAL_ZONES];
707     int choice;
708     /* Load assignment data only once */
709     loadOriginalData(original);
710     do
711     {
712         printf("\n\n");
713         printf("===== EMERGENCY POTABLE WATER ALLOCATION\n");
714         printf("=====");
715         printf("1. Case 1 - Original Dataset\n");
716         printf("2. Case 2 - Enough Water\n");
717         printf("3. Case 3 - Below Minimum Requirement\n");
718         printf("4. Case 4 - Equal Priority\n");
719         printf("5. Case 5 - Increased Distribution Loss\n");
720         printf("6. Case 6 - Increased Waiting Period\n");
721         printf("7. Case 7 - Independent Test\n");
```

Logs & others

Debug or continue program (depends on context) C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

<global> main0: int

Management
Projects
Workspace

```
721 printf("6. Case 6 - Increased Waiting Period\n");
722 printf("7. Case 7 - Independent Test\n");
723 printf("8. Search a Zone\n");
724 printf("9. Find Highest Unresolved Requirement\n");
725 printf("10. Exit\n");
726 printf("Enter your choice: ");
727 scanf("%d", &choice);
728 switch (choice)
729 {
730     case 1:
731         case1(original);
732         break;
733     case 2:
734         case2(original);
735         break;
736     case 3:
737         case3(original);
738         break;
739     case 4:
740         case4(original);
741         break;
742     case 5:
743         case5(original);
744         break;
745     case 6:
746         case6(original);
747         break;
748     case 7:
749         case7(original);
750         break;
751     case 8:
752 }
```

Logs & others

C:\Users\user\OneDrive\Documents\1384==.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 779, Col 2, Pos 25455 Insert Read/Write default

1384==.c - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

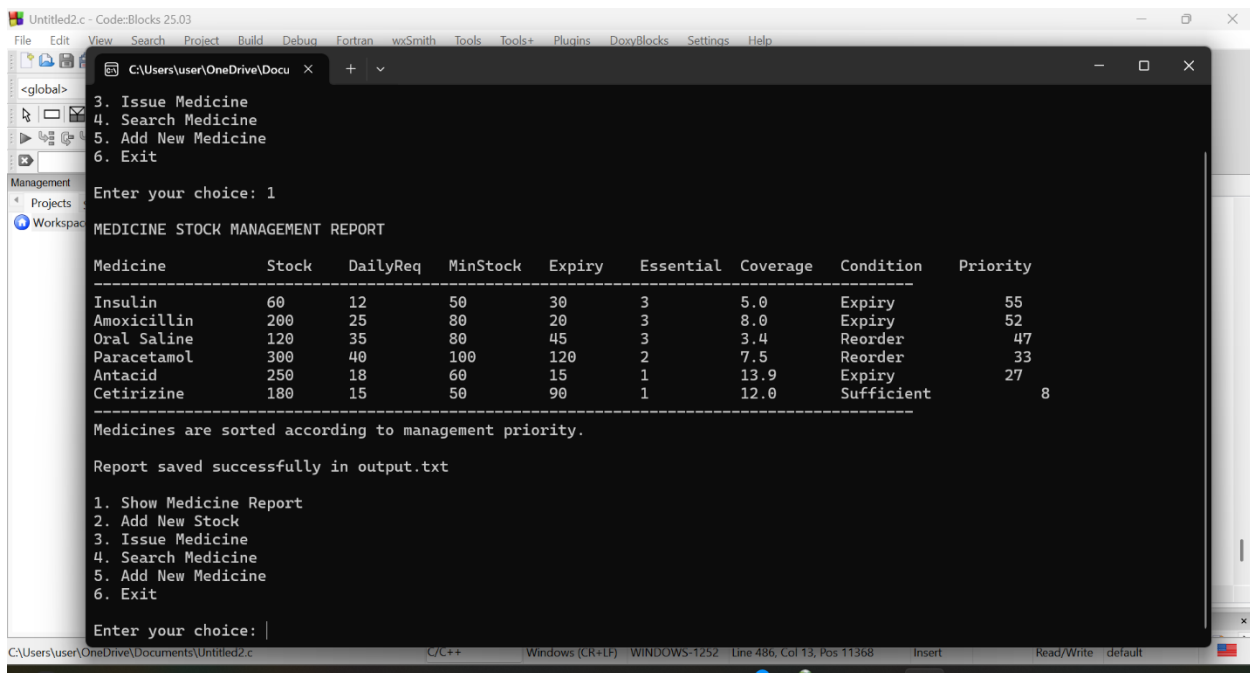
<global> main0: int

Management
Projects
Workspace

```
750 break;
751 case 8:
752 {
753     struct Zone zone(TOTAL_ZONES);
754     int i;
755     for (i = 0; i < TOTAL_ZONES; i++)
756         zone[i] = original[i];
757     runAllocation(zone, 13500);
758     searchZone(zone);
759     break;
760 }
761 case 9:
762 {
763     struct Zone zone(TOTAL_ZONES);
764     int i;
765     for (i = 0; i < TOTAL_ZONES; i++)
766         zone[i] = original[i];
767     runAllocation(zone, 13500);
768     findHighestShortage(zone);
769     break;
770 }
771 case 10:
772     printf("Program ended.\n");
773     break;
774 default:
775     printf("Invalid choice. Please try again.\n");
776 } while (choice != 10);
777 return 0;
778 }
779
780 }
```

Logs & others

6. Testing and Validation



```
Untitled2.c - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
C:\Users\user\OneDrive\Docu x + v
<global>
3. Issue Medicine
4. Search Medicine
5. Add New Medicine
6. Exit
Enter your choice: 1
MEDICINE STOCK MANAGEMENT REPORT
Medicine      Stock  DailyReq  MinStock  Expiry  Essential  Coverage  Condition  Priority
-----
Insulin        60     12        50        30       3         5.0      Expiry      55
Amoxicillin    200     25        80        20       3         8.0      Expiry      52
Oral Saline    120     35        80        45       3         3.4      Reorder     47
Paracetamol    300     40       100       120      2         7.5      Reorder     33
Antacid        250     18        60        15       1        13.9     Expiry      27
Cetirizine     180     15        50        90       1        12.0     Sufficient   8
-----
Medicines are sorted according to management priority.
Report saved successfully in output.txt
1. Show Medicine Report
2. Add New Stock
3. Issue Medicine
4. Search Medicine
5. Add New Medicine
6. Exit
Enter your choice: |
```

C:\Users\user\OneDrive\Documents\Untitled2.c C/C++ Windows (CR+LF) WINDOWS-1252 Line 486, Col 13, Pos 11368 Insert Read/Write default

```
C:\Users\user\OneDrive\Docu  X + v

=====
EMERGENCY POTABLE WATER ALLOCATION
=====
1. Case 1 - Original Dataset
2. Case 2 - Enough Water
3. Case 3 - Below Minimum Requirement
4. Case 4 - Equal Priority
5. Case 5 - Increased Distribution Loss
6. Case 6 - Increased Waiting Period
7. Case 7 - Independent Test
8. Search a Zone
9. Find Highest Unresolved Requirement
10. Exit

Enter your choice: 2

===== CASE 2 =====
Enough Water for Complete Requested Quantity
Available Water: 22194.00 L

=====
ALLOCATION RESULT
=====
ID      Zone      Request  Minimum  Priority  Allocated  Delivered  Shortage  Status
-----
Z04  Ward 5      3800     1800     13.80     4318       3800       0     MINIMUM
Z03  Ward 3      4500     2000     13.20     4891       4500       0     FULL
Z01  Municipal Hospital 4000     3000     11.80     4082       4000       0     FULL
Z02  Central Flood Shelter 3500     2500     11.60     3646       3500       0     FULL
Z05  School Emergency Shelter 2600     1600     11.50     2737       2600       0     FULL
Z06  Fire and Emergency Service 2000     1500     10.90     2020       2000       0     FULL
=====

Priority Order:
1. Z04 - Ward 5 (Priority 13.80)
2. Z03 - Ward 3 (Priority 13.20)
3. Z01 - Municipal Hospital (Priority 11.80)
4. Z02 - Central Flood Shelter (Priority 11.60)
5. Z05 - School Emergency Shelter (Priority 11.50)
6. Z06 - Fire and Emergency Service (Priority 10.90)

Summary
-----
Total Requested Water      : 20400.00 L
Total Allocated Water      : 21694.00 L
Total Effective Water Delivered : 20400.00 L
Total Unresolved Requirement : 0.00 L
Remaining Water            : 500.00 L

Service Conditions
-----
Full Requirement Satisfied : 5 zone(s)
Minimum Requirement Satisfied: 1 zone(s)
Below Minimum              : 0 zone(s)
Completely Unserved        : 0 zone(s)
```

```
C:\Users\user\OneDrive\Docu X + v

Enter your choice: 3

===== CASE 3 =====
Enter available water in litres: 100

=====
ALLOCATION RESULT
=====
ID Zone Request Minimum Priority Allocated Delivered Shortage Status
Z04 Ward 5 3800 1800 13.80 100 88 3712 BELOW MIN
Z03 Ward 3 4500 2000 13.20 0 0 4500 UNSERVED
Z01 Municipal Hospital 4000 3000 11.80 0 0 4000 UNSERVED
Z02 Central Flood Shelter 3500 2500 11.60 0 0 3500 UNSERVED
Z05 School Emergency Shelter 2600 1600 11.50 0 0 2600 UNSERVED
Z06 Fire and Emergency Service 2000 1500 10.90 0 0 2000 UNSERVED

Priority Order:
1. Z04 - Ward 5 (Priority 13.80)
2. Z03 - Ward 3 (Priority 13.20)
3. Z01 - Municipal Hospital (Priority 11.80)
4. Z02 - Central Flood Shelter (Priority 11.60)
5. Z05 - School Emergency Shelter (Priority 11.50)
6. Z06 - Fire and Emergency Service (Priority 10.90)

Summary
Total Requested Water : 20400.00 L
Total Allocated Water : 100.00 L
Total Effective Water Delivered : 88.00 L
Total Unresolved Requirement : 20312.00 L
Remaining Water : 0.00 L

Service Conditions
Full Requirement Satisfied : 0 zone(s)
Minimum Requirement Satisfied: 0 zone(s)
Below Minimum : 1 zone(s)
Completely Unservd : 5 zone(s)
```

```
C:\Users\user\OneDrive\Docu X + v

Default: C:\Users\user\OneDrive\Documents
\1384==.exe
ctrl+alt+1 priority.

=====
ALLOCATION RESULT
=====
ID Zone Request Minimum Priority Allocated Delivered Shortage Status
Z03 Ward 3 4000 2000 14.00 4444 4000 0 FULL
Z04 Ward 5 4000 2000 14.00 4444 4000 0 FULL
Z01 Municipal Hospital 4000 3000 11.80 4082 4000 0 FULL
Z02 Central Flood Shelter 3500 2500 11.60 529 508 2992 BELOW MIN
Z05 School Emergency Shelter 2600 1600 11.50 0 0 2600 UNSERVED
Z06 Fire and Emergency Service 2000 1500 10.90 0 0 2000 UNSERVED

Priority Order:
1. Z03 - Ward 3 (Priority 14.00)
2. Z04 - Ward 5 (Priority 14.00)
3. Z01 - Municipal Hospital (Priority 11.80)
4. Z02 - Central Flood Shelter (Priority 11.60)
5. Z05 - School Emergency Shelter (Priority 11.50)
6. Z06 - Fire and Emergency Service (Priority 10.90)

Summary
Total Requested Water : 20100.00 L
Total Allocated Water : 13500.00 L
Total Effective Water Delivered : 12508.30 L
Total Unresolved Requirement : 7591.70 L
Remaining Water : 0.00 L

Service Conditions
Full Requirement Satisfied : 3 zone(s)
Minimum Requirement Satisfied: 0 zone(s)
Below Minimum : 1 zone(s)
Completely Unservd : 2 zone(s)
```



```
C:\Users\user\OneDrive\Docu  X  +  v

Enter your choice: 5

===== CASE 5 =====
Residential zones are: Z03 and Z04
Enter residential Zone ID: Z03
Current distribution loss: 8.00%
Enter new distribution loss (%): 10

New loss of Z03 = 10.00%

=====
ALLOCATION RESULT
=====
ID   Zone           Request  Minimum  Priority  Allocated  Delivered  Shortage  Status
-----
Z04  Ward 5           3800     1800     13.80    4318      3800      0         MINIMUM
Z03  Ward 3           4500     2000     13.00    5000      4500      0         FULL
Z01  Municipal Hospital 4000     3000     11.80    4082      4000      0         FULL
Z02  Central Flood Shelter 3500     2500     11.60    100       96        3404     BELOW MIN
Z05  School Emergency Shelter 2600     1600     11.50    0          0         2600     UNSERVED
Z06  Fire and Emergency Service 2000     1500     10.90    0          0         2000     UNSERVED
=====

Priority Order:
1. Z04 - Ward 5 (Priority 13.80)
2. Z03 - Ward 3 (Priority 13.00)
3. Z01 - Municipal Hospital (Priority 11.80)
4. Z02 - Central Flood Shelter (Priority 11.60)
5. Z05 - School Emergency Shelter (Priority 11.50)
6. Z06 - Fire and Emergency Service (Priority 10.90)

Summary
-----
Total Requested Water      : 20400.00 L
Total Allocated Water      : 13500.00 L
Total Effective Water Delivered : 12396.18 L
Total Unresolved Requirement : 8003.82 L
Remaining Water            : 0.00 L

Service Conditions
-----
Full Requirement Satisfied : 2 zone(s)
Minimum Requirement Satisfied: 1 zone(s)
Below Minimum              : 1 zone(s)
Completely Unserved        : 2 zone(s)
```

```
C:\Users\user\OneDrive\Docu  X  +  v

Enter your choice: 6

===== CASE 6 =====
Previously underserved zones from Case 1:
Z02 - Central Flood Shelter
Z05 - School Emergency Shelter
Z06 - Fire and Emergency Service

Enter Zone ID from the underserved list: Z02
Current waiting cycles: 1
Enter new waiting cycles: 3

Waiting cycles of Z02 changed to 3.

=====
ALLOCATION RESULT
=====
ID   Zone           Request  Minimum  Priority  Allocated  Delivered  Shortage  Status
-----
Z02  Central Flood Shelter 3500     2500     15.60    3646      3500      0         FULL
Z04  Ward 5           3800     1800     13.80    4318      3800      0         MINIMUM
Z03  Ward 3           4500     2000     13.20    4891      4500      0         FULL
Z01  Municipal Hospital 4000     3000     11.80    645       632       3368     BELOW MIN
Z05  School Emergency Shelter 2600     1600     11.50    0          0         2600     UNSERVED
Z06  Fire and Emergency Service 2000     1500     10.90    0          0         2000     UNSERVED
=====

Priority Order:
1. Z02 - Central Flood Shelter (Priority 15.60)
2. Z04 - Ward 5 (Priority 13.80)
3. Z03 - Ward 3 (Priority 13.20)
4. Z01 - Municipal Hospital (Priority 11.80)
5. Z05 - School Emergency Shelter (Priority 11.50)
6. Z06 - Fire and Emergency Service (Priority 10.90)

Summary
-----
Total Requested Water      : 20400.00 L
Total Allocated Water      : 13500.00 L
Total Effective Water Delivered : 12431.79 L
Total Unresolved Requirement : 7968.21 L
Remaining Water            : 0.00 L

Service Conditions
-----
Full Requirement Satisfied : 2 zone(s)
Minimum Requirement Satisfied: 1 zone(s)
```

Enter your choice: 7

===== CASE 7 =====

Independent Test Case

Enter any available water quantity: 6000

=====								
ALLOCATION RESULT								
ID	Zone	Request	Minimum	Priority	Allocated	Delivered	Shortage	Status
Z04	Ward 5	3800	1800	13.80	4318	3800	0	MINIMUM
Z03	Ward 3	4500	2000	13.20	1682	1547	2953	BELOW MIN
Z01	Municipal Hospital	4000	3000	11.80	0	0	4000	UNSERVED
Z02	Central Flood Shelter	3500	2500	11.60	0	0	3500	UNSERVED
Z05	School Emergency Shelter	2600	1600	11.50	0	0	2600	UNSERVED
Z06	Fire and Emergency Service	2000	1500	10.90	0	0	2000	UNSERVED
=====								

Priority Order:

1. Z04 - Ward 5 (Priority 13.80)
2. Z03 - Ward 3 (Priority 13.20)
3. Z01 - Municipal Hospital (Priority 11.80)
4. Z02 - Central Flood Shelter (Priority 11.60)
5. Z05 - School Emergency Shelter (Priority 11.50)
6. Z06 - Fire and Emergency Service (Priority 10.90)

Summary

Total Requested Water : 20400.00 L
Total Allocated Water : 6000.00 L
Total Effective Water Delivered : 5347.27 L
Total Unresolved Requirement : 15052.73 L
Remaining Water : 0.00 L

Service Conditions

Full Requirement Satisfied : 0 zone(s)
Minimum Requirement Satisfied: 1 zone(s)
Below Minimum : 1 zone(s)
Completely Unserved : 4 zone(s)

Enter your choice: 8

Enter Zone ID: Z01

Zone Found

Zone ID : Z01
Service Zone : Municipal Hospital
Type : Hospital
Requested : 4000.00 L
Minimum : 3000.00 L
Loss : 2.00 %
Waiting Cycles : 0
Priority : 11.80
Allocated : 4081.63 L
Loss Amount : 81.63 L
Effective Water : 4000.00 L
Shortage : 0.00 L
Condition : FULL

```
Enter your choice: 9
```

```
Highest Unresolved Requirement
```

```
-----  
Zone ID   : Z02  
Zone      : Central Flood Shelter  
Shortage  : 3299.47 L
```

```
=====
```

EMERGENCY POTABLE WATER ALLOCATION

```
=====
```

1. Case 1 - Original Dataset
2. Case 2 - Enough Water
3. Case 3 - Below Minimum Requirement
4. Case 4 - Equal Priority
5. Case 5 - Increased Distribution Loss
6. Case 6 - Increased Waiting Period
7. Case 7 - Independent Test
8. Search a Zone
9. Find Highest Unresolved Requirement
10. Exit

7. Limitations

The proposed solution rests on a number of assumptions and carries corresponding limitations that should be understood when interpreting its output.

- The distribution-loss percentage recorded for a zone is treated as a fixed characteristic of its supply route for the current cycle; the program does not model losses that vary with the quantity released, with weather conditions, or with the time elapsed during distribution.
- The previous waiting cycles figure is treated as an externally supplied count reflecting the zone's history prior to the current cycle; the program does not itself track or accumulate waiting cycles across multiple runs unless it is extended to persist this information between executions.
- The priority-scoring weights assigned to zone-type criticality, minimum-requirement urgency and waiting cycles represent one reasonable, explained approach among several that could satisfy the assignment brief, which explicitly permits students to adopt a different, justified strategy; another reasonable strategy would be expected to produce broadly similar, though not necessarily identical, allocations on the supplied dataset.
- The two-phase allocation assumes that guaranteeing every zone's minimum requirement before extending any zone toward its full request is the appropriate ordering of objectives; a municipality that instead wished to priorities fully satisfying its most critical zones first, even at the expense of the minimum requirement of lower-priority zones, would need a different allocation structure.
- The dataset size assumed for demonstration is small, and the sorting technique selected in Section 5 is chosen accordingly; a much larger number of service zones would warrant reconsideration of the selected technique.

8. Conclusion

This assignment required the design of a structured, C-language solution to an emergency resource-allocation problem in which the total demand for potable water exceeds the available supply, and in which

the zones requesting water differ in criticality, prior waiting history and distribution efficiency. The proposed solution assigns every zone a priority score that combines its type criticality, its minimum-requirement urgency and its previous waiting cycles, orders the zones accordingly, and allocates the available water in two phases that first guarantee, as far as the available water permits, the minimum requirement of every zone and then extend the remaining water toward the full requested quantity of each zone in priority order. Distribution loss is applied consistently when converting a target delivered quantity into the corresponding release quantity, rather than being allowed to distort the priority ranking itself, addressing the specific concern raised in the assignment brief regarding the interaction between distribution efficiency and social requirement. The design keeps every derived value attached to the zone record it describes, computes each derived value once, and is intended to remain correct when the supplied requested quantities, minimum requirements, loss percentages, waiting cycles or available quantity are changed. Sections 6 and 7 of this report, concerning the implementation and its testing, are to be completed separately alongside the submitted source code and test evidence.

9. References

[1] Course materials and assignment brief, "Assignment on Structured Programming LAB — Emergency Potable Water Allocation for a Flood-Affected Municipality," CSC 1384 Structured Programming LAB, Summer 2026.